

Aarya Arban

T11 – 05

Assignment No. 11

Aim:

To write a program using AJAX for user validation and to show the result on the same page below the submit button.

LO Mapping: LO4: Develop Rich Internet applications using AJAX

Theory:

What is AJAX?

AJAX is a technique used in web development to create interactive and dynamic web applications. It stands for Asynchronous JavaScript and XML, although XML is not strictly required and other data formats such as JSON (JavaScript Object Notation) are commonly used.

Principles of AJAX:

1. Asynchronous Communication:

- AJAX allows web pages to make asynchronous requests to the server without needing to reload the entire page.
- Asynchronous requests can be made in the background while the user interacts with the page, providing a smoother user experience.

2. Client-Side Scripting:

- AJAX relies on client-side scripting languages such as JavaScript to handle asynchronous requests and update the page content dynamically.

3. Server-Side Processing:

- AJAX requests are typically handled by server-side scripts or APIs

that process the request, perform any necessary operations (such as database queries or calculations), and return a response.

4. Data Exchange Formats:

- AJAX requests can exchange data with the server in various formats, including XML, JSON, plain text, or HTML.
- JSON (JavaScript Object Notation) is commonly used due to its lightweight and easy-to-parse nature.

5. DOM Manipulation:

- Once the response is received from the server, the client-side script can dynamically update the DOM (Document Object Model) to reflect the changes without requiring a page reload.
- This enables creating dynamic user interfaces and real-time updates.

Usage of AJAX:

1. Form Submission:

- AJAX is often used to submit form data to the server without reloading the entire page.
- This allows for a more seamless user experience and faster response times.

2. Data Retrieval:

- AJAX can retrieve data from the server in response to user actions, such as clicking a button or scrolling a page.
- This data can be used to dynamically update the page content, such as fetching new articles in a news feed or loading additional product listings in an e-commerce website.

3. Real-Time Updates:

- AJAX enables real-time updates to web applications by periodically fetching new data from the server and updating the page content accordingly.
- This is commonly used in chat applications, social media feeds, and live sports scores.

4. Autosave Functionality:

- AJAX can be used to implement autosave functionality in web forms, where form data is periodically sent to the server in the background to prevent data loss in case of browser crashes or accidental navigation away from the page.

5. Interactive Web Applications:

- AJAX is essential for creating interactive and responsive web applications, where user actions trigger asynchronous requests to the server, and the page content is updated dynamically without full page reloads.

Advantages of AJAX:

1. Improved User Experience:

- AJAX enables smoother and more responsive web applications by reducing page reloads and providing real-time updates.

2. Reduced Server Load:

- Asynchronous requests allow for selective updates of page content, reducing the amount of data transferred between the client and server and lowering server load.

3. Faster Performance:

- By fetching data asynchronously, AJAX requests can improve performance by fetching only the necessary data and updating the page dynamically, without waiting for the entire page to reload.

4. Enhanced Interactivity:

- AJAX enables rich and interactive user interfaces by allowing for seamless updates to page content based on user actions.

5. Cross-Browser Compatibility:

- AJAX is supported by all major web browsers, making it a reliable and widely adopted technique for web development.

Code:

```
<body>
<div class="container">
  <h2>User Validation</h2>
  <form id="validationForm">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required><br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required><br>
    <button type="submit" id="submitBtn">Submit</button>
  </form>

  <div id="validationResult"></div>
</div>

<script>
document.getElementById('validationForm').addEventListener('submit', function(event){
  event.preventDefault(); // Prevent form submission

  var username = document.getElementById('username').value;
  var password = document.getElementById('password').value;

  // Basic client-side validation
  if(username.trim() === "" || password.trim() === "") {
    document.getElementById('validationResult').innerHTML = "<p id='errorMsg'>Please enter both username and password.</p>";
    return;
  }

  // AJAX request to simulate server-side validation
  var xhr = new XMLHttpRequest();
  xhr.open('GET', 'response.json', true); // Point to a JSON file containing response
  xhr.setRequestHeader('Content-Type', 'application/json');
```

```
<script>
document.getElementById('validationForm').addEventListener('submit', function(event){
  // AJAX request to simulate server-side validation
  var xhr = new XMLHttpRequest();
  xhr.open('GET', 'response.json', true); // Point to a JSON file containing response
  xhr.setRequestHeader('Content-Type', 'application/json');

  xhr.onload = function() {
    if (xhr.status === 200) {
      var response = JSON.parse(xhr.responseText);
      if (response.validCredentials.hasOwnProperty(username) && response.validCredentials[username] === password) {
        document.getElementById('validationResult').innerHTML = "<p id='successMsg'>Username and password are valid!</p>";
      } else {
        document.getElementById('validationResult').innerHTML = "<p id='errorMsg'>Invalid username or password. Please try again.</p>";
      }
    } else {
      console.log('Request failed. Status: ' + xhr.status);
    }
  };
  xhr.send();
});
</script>

</body>
</html>
```

Output:

The image displays two sequential screenshots of a web browser window, illustrating the output of a user validation process. Both screenshots show a form titled "User Validation" with fields for "Username:" and "Password:", a green "Submit" button, and a feedback message.

Top Screenshot (Invalid Login):

- Username: admin
- Password: *****
- Submit button
- Feedback message: Invalid username or password. Please try again.

Bottom Screenshot (Valid Login):

- Username: admin
- Password: *****
- Submit button
- Feedback message: Username and password are valid!

Conclusion:

AJAX is a powerful technique that revolutionized web development by enabling the creation of interactive, dynamic, and responsive web applications. By leveraging asynchronous communication between the client and server, AJAX provides a smoother user experience, faster performance,

and enhanced interactivity, making it an essential tool for modern web development.